

STAR ESP32 Firmware Implementation Primer

ESP32-S3 Motor Control and Sensor Integration Guide

STAR Development Team

December 2025

Contents

1	Project Overview	3
1.1	Purpose	3
1.2	Current Implementation Status	3
2	Architecture Overview	3
2.1	Dependency Inversion Principle (DIP)	3
2.2	Directory Structure	3
2.3	Coding Style and Conventions	4
3	Motor Control Architecture	4
3.1	Motor Control Refactoring Plan	4
3.1.1	lib/star_motor (Proposed)	4
3.1.2	lib/star_encoder (Proposed)	5
3.1.3	lib/star_pid (Proposed)	5
3.2	Core Communication Interfaces	6
3.2.1	SPI Peripheral Driver	6
4	Extending the Bus Abstraction Layer	7
4.1	Task 1: Re-create GPIO Bus Driver	7
4.1.1	Modify Type Definition Files	7
4.1.2	Create GPIO Driver	8
4.1.3	Update Bus Config Factory	8
4.1.4	Update Build System	8
4.2	Task 2: Add ADC Bus Protocol	8
4.2.1	Modify Type Definition Files	8
4.2.2	Create ADC Driver	9
4.2.3	Update Bus Config Factory	9
4.2.4	Update Build System	9
4.3	Task 3: Implement Unified Bus Interface	9
4.3.1	Add Generic Functions to Bus Manager	9
4.4	Task 4: Final Bus Library Cleanup	10

5	Testing Plan	10
5.1	Unit Tests	10
5.2	Integration Tests	11
5.3	Acceptance Criteria	11
6	Build and Flash Instructions	11
6.1	Prerequisites	11
6.2	Build Commands	11
6.3	Configuration	11
7	References	12

1 Project Overview

1.1 Purpose

The `star-esp32-firmware` project provides real-time motor control, sensor integration, and communication with the Raspberry Pi 5 gateway for the STAR robot platform. It runs on the ESP32-S3 microcontroller using FreeRTOS.

1.2 Current Implementation Status

Component	Status	Notes
STAR Framework Core	Complete	DIP architecture, interfaces
Bus Abstraction Layer	Complete	I2C, SPI, UART, GPIO, ADC, 1-Wire
Error Handling	Complete	Retry logic with exponential backoff
Pin Validation	Complete	GPIO conflict detection
Motor Control (MCPWM)	Complete	<code>lib/star_motor</code>
Motor Driver Integration	Complete	<code>lib/star_drv8243</code> (DRV8243 H-bridge)
PID Controller	Complete	<code>lib/star_pid</code>
Encoder Reading	Complete	<code>lib/star_encoder</code> (PCNT-based)
Temperature Sensor	Complete	<code>lib/star_sensor_ds18b20</code> (1-Wire)
Battery Management	Complete	<code>lib/star_bms_bq7850</code>
SPI Peripheral (Pi5 comm)	Complete	Low-level communication layer ready
Application Code	TODO	Integration example in <code>src/main.c</code>

Table 1: Implementation Status Summary

2 Architecture Overview

2.1 Dependency Inversion Principle (DIP)

The firmware follows SOLID principles with emphasis on Dependency Inversion:

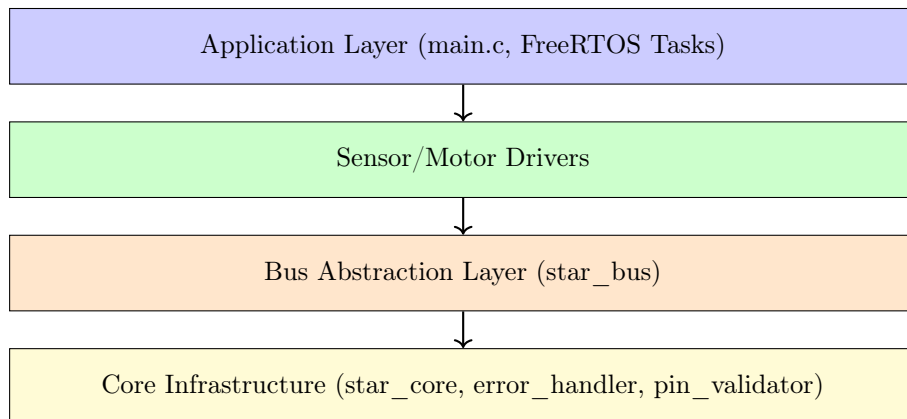


Figure 1: STAR Firmware Layered Architecture

2.2 Directory Structure

```

1 star-esp32-firmware/
2 |-- src/                                # Application entry point
3 |
4 |-- lib/
5 |   |-- star_core/                     # Abstract interfaces (DIP foundation)
6 |   |-- star_error_handler/           # Error handling with retry logic
7 |   |-- star_pin_validator/           # GPIO conflict detection
8 |   |-- star_bus/                      # Unified bus abstraction
9 |   |   |                               # (I2C, SPI, UART, GPIO, ADC, 1-Wire)
10 |   |-- star_motor/                   # Brushed DC motor control (MCPWM)
11 |   |-- star_drv8243/                 # DRV8243 H-bridge driver integration
12 |   |-- star_encoder/                 # Quadrature encoder (PCNT-based)
13 |   |-- star_pid/                     # PID controller
14 |   |-- star_sensor_ds18b20/          # DS18B20 temperature sensor (1-Wire)
15 |   +-- star_bms_bq7850/              # BQ7850 battery management system
16 |
17 |-- platformio.ini                    # Build configuration
18 +-- sdkconfig.*                       # ESP-IDF configs

```

2.3 Coding Style and Conventions

To ensure long-term maintainability, consistency, and readability of the firmware, this project adheres to a strict set of coding conventions. All contributions must follow the rules outlined in the official project style guide.

The primary source of truth for all coding style is the docs/style_guide.pdf document and the CLAUDE.md architecture guide.

Adherence to this style guide is not optional; it is a critical requirement for all development within this project.

3 Motor Control Architecture

3.1 Motor Control Components

The STAR firmware implements a modular motor control stack with clear separation of concerns:

3.1.1 lib/star_motor - Low-Level MCPWM Control

Provides hardware abstraction for brushed DC motor control using ESP32's MCPWM peripheral.

Key Features:

- Bidirectional PWM control (-100% to +100% duty cycle)
- Automatic dead-time insertion for H-bridge safety
- Configurable PWM frequency (default: 20 kHz)
- Optional fault input for overcurrent protection
- Brake and coast modes

No bus dependency - Direct hardware peripheral control.

Listing 1: Motor Driver Interface

```

1  typedef struct {
2      int group_id;
3      uint32_t timer_resolution_hz;
4      uint32_t pwm_freq_hz;
5      gpio_num_t pin_pwm_a;
6      gpio_num_t pin_pwm_b;
7      uint32_t dead_time_ns;
8      gpio_num_t fault_pin;
9      bool invert_pwm;
10 } star_motor_config_t;
11
12 esp_err_t star_motor_init(star_motor_handle_t* handle,
13                           const star_motor_config_t* config);
14 esp_err_t star_motor_set_duty(star_motor_handle_t* handle, float duty);
15 esp_err_t star_motor_stop(star_motor_handle_t* handle, bool brake);

```

3.1.2 lib/star_drv8243 - DRV8243 Motor Driver Integration

Integrates motor control with current sensing and fault detection via bus manager.

Key Features:

- H-bridge motor control (one brushed DC motor)
- Current sensing via IPROPI pin (ADC)
- Fault detection via nFAULT pin (GPIO)
- Configurable current limiting
- PH/EN control mode (PWM + direction)
- PWM frequency: up to 25 kHz

Uses **bus manager** for ADC (current sensing) and GPIO (fault detection).

3.1.3 lib/star_encoder - Quadrature Encoder

Handles position and velocity feedback using ESP32 PCNT peripheral.

Key Features:

- Hardware-accelerated quadrature decoding
- Position tracking with signed 32-bit counter
- Velocity calculation (RPM)
- Configurable glitch filter
- Counter limits and overflow detection

No bus dependency - Direct PCNT peripheral access.

3.1.4 lib/star_pid - PID Controller

Generic, stateless PID controller for closed-loop control.

Key Features:

- Standard PID algorithm with K_p , K_i , K_d gains
- Configurable output limits
- Integral anti-windup
- Runtime gain tuning
- State reset capability

No dependencies - Pure algorithm, hardware-agnostic.

Listing 2: PID Controller Interface

```

1 typedef struct {
2     float kp, ki, kd;
3     float output_min, output_max;
4     float integral_min, integral_max;
5 } star_pid_config_t;
6
7 esp_err_t star_pid_init(star_pid_handle_t* handle,
8                         const star_pid_config_t* config);
9 esp_err_t star_pid_compute(star_pid_handle_t* handle,
10                            float setpoint, float measured,
11                            float dt, float* output);
12 esp_err_t star_pid_reset(star_pid_handle_t* handle);

```

Recommended Tuning (adjust experimentally):

- $K_p = 1.0$, $K_i = 0.5$, $K_d = 0.1$
- Loop rate: 1000 Hz
- Integral limit: ± 50

3.2 Core Communication Interfaces

3.2.1 SPI Peripheral Driver

The firmware implements SPI peripheral mode for communication with the Raspberry Pi 5.

- **Mode:** Peripheral on SPI2 (HSPI)
- **Protocol:** Custom binary packet structure
- **Performance:** DMA is used for efficient, non-blocking transfers

SPI Pin Mapping:

Signal	Pi5 GPIO	ESP32 GPIO
COPI	GPIO11	GPIO13
CIP0	GPIO9	GPIO12
SCLK	GPIO11	GPIO14
CS	GPIO8	GPIO15

Command Protocol:

Listing 3: SPI Command Protocol

```

1  // Command byte definitions
2  #define CMD_SET_VELOCITY      0x01  // Set wheel velocities
3  #define CMD_EMERGENCY_STOP    0x02  // Immediate stop
4  #define CMD_GET_ENCODERS      0x03  // Read encoder counts
5  #define CMD_GET_TELEMETRY     0x06  // Battery, temp, status
6
7  // Packet format: [CMD (1)] [LENGTH (1)] [DATA (N)] [CRC8 (1)]
8  typedef struct __attribute__((packed)) {
9      uint8_t command;
10     uint8_t length;
11     uint8_t data[32];
12     uint8_t crc8;
13 } spi_packet_t;
14
15 // Velocity command payload
16 typedef struct __attribute__((packed)) {
17     float left_velocity;    // m/s
18     float right_velocity;   // m/s
19     uint32_t timestamp;     // ms since boot
20 } velocity_cmd_t;

```

4 Complete Motor Control System Example

4.1 Closed-Loop Velocity Control

The following example demonstrates a complete closed-loop motor control system integrating all motor control components with thermal monitoring:

Listing 4: Complete Motor Control System

```

1  #include "star_drv8243.h"
2  #include "star_encoder.h"
3  #include "star_pid.h"
4  #include "star_sensor_ds18b20.h"
5
6  // === Setup Phase ===
7
8  // 1. Initialize bus manager
9  star_bus_manager_t bus_manager;
10 star_bus_manager_init(&bus_manager, "main", NULL, NULL);
11
12 // 2. Create buses
13 star_bus_config_t* gpio_bus = star_bus_config_create_gpio("gpio_bus");

```

```

14 star_bus_config_t* adc_bus = star_bus_config_create_adc(
15     "adc_bus", ADC_UNIT_1, ADC_ATTEN_DB_12, ADC_BITWIDTH_12);
16 star_bus_config_t* onewire_bus = star_bus_config_create_onewire(
17     "temp_bus", GPIO_NUM_4, false);
18
19 star_bus_manager_add_bus(&bus_manager, gpio_bus);
20 star_bus_manager_add_bus(&bus_manager, adc_bus);
21 star_bus_manager_add_bus(&bus_manager, onewire_bus);
22
23 // 3. Initialize motor driver
24 star_drv8243_handle_t motor;
25 star_drv8243_config_t motor_cfg = {
26     .bus_manager = &bus_manager,
27     .gpio_bus_name = "gpio_bus",
28     .adc_bus_name = "adc_bus",
29     .pin_pwm_ph = GPIO_NUM_25,
30     .pin_pwm_en = GPIO_NUM_26,
31     .pin_ipropi = ADC_CHANNEL_6,
32     .pin_nfault = GPIO_NUM_35,
33     .pwm_freq_hz = 20000,
34     .current_limit_ma = 2000,
35 };
36 star_drv8243_init(&motor, &motor_cfg);
37
38 // 4. Initialize encoder
39 star_encoder_handle_t encoder;
40 star_encoder_config_t enc_cfg = {
41     .pcnt_unit = PCNT_UNIT_0,
42     .pin_a = GPIO_NUM_32,
43     .pin_b = GPIO_NUM_33,
44     .filter_value = 1000,
45     .high_limit = 10000,
46     .low_limit = -10000,
47 };
48 star_encoder_init(&encoder, &enc_cfg);
49
50 // 5. Initialize PID controller
51 star_pid_handle_t pid;
52 star_pid_config_t pid_cfg = {
53     .kp = 1.0f, .ki = 0.5f, .kd = 0.1f,
54     .output_min = -100.0f, .output_max = 100.0f,
55     .integral_min = -50.0f, .integral_max = 50.0f,
56 };
57 star_pid_init(&pid, &pid_cfg);
58
59 // 6. Initialize temperature sensor
60 star_ds18b20_handle_t temp_sensor;
61 star_ds18b20_config_t temp_cfg = {
62     .bus_manager = &bus_manager,
63     .bus_name = "temp_bus",
64     .resolution = k_star_ds18b20_resolution_12_bit,
65     .use_rom = false,
66 };
67 star_sensor_ds18b20_init(&temp_sensor, &temp_cfg);

```



```

68
69 // === Control Loop (1000Hz) ===
70
71 float setpoint_rpm = 100.0f;
72 float dt = 0.001f; // 1ms
73
74 while (1) {
75     // Read encoder velocity
76     float velocity_rpm;
77     star_encoder_get_velocity_rpm(&encoder, dt * 1000, 500, &velocity_rpm)
78     ;
79
80     // Compute PID output
81     float pid_output;
82     star_pid_compute(&pid, setpoint_rpm, velocity_rpm, dt, &pid_output);
83
84     // Apply to motor
85     star_drv8243_set_speed(&motor, pid_output);
86
87     // Monitor current and temperature
88     float current_ma, temp_c;
89     star_drv8243_read_current(&motor, &current_ma);
90     star_sensor_ds18b20_read_temp(&temp_sensor, &temp_c);
91
92     // Safety checks
93     bool fault;
94     star_drv8243_get_fault_status(&motor, &fault);
95     if (fault || temp_c > 85.0f) {
96         star_drv8243_stop(&motor, true); // Emergency brake
97     }
98
99     vTaskDelay(pdMS_TO_TICKS(1)); // 1ms delay
100 }

```

4.2 Architecture Benefits

The modular design provides:

- **Clear separation of concerns:** Each library has a single, well-defined purpose
- **Testability:** Low-level components (motor, encoder, PID) have no dependencies
- **Reusability:** PID and encoder libraries work with any motor driver
- **Safety:** Current limiting and fault detection at the integration layer
- **Flexibility:** Easy to swap motor drivers or add new control algorithms

5 Testing Plan

5.1 Unit Tests

1. **PID Controller:** Test with simulated plant, verify convergence

2. **Encoder Driver:** Test count accuracy, overflow handling
3. **Motor Driver:** Test PWM generation, dead-time
4. **SPI Protocol:** Test packet parsing, CRC validation

5.2 Integration Tests

1. **Motor + Encoder:** Closed-loop step response test
2. **SPI + Motor:** Command-to-motion latency measurement
3. **Full System:** Velocity tracking accuracy at various speeds

5.3 Acceptance Criteria

Metric	Target
PID loop rate	1000 Hz $\pm$ 1%
Velocity tracking error	< 5% at steady state
Command latency (SPI)	< 1ms
Emergency stop time	< 10ms

6 Build and Flash Instructions

6.1 Prerequisites

```

1 # Install PlatformIO
2 pip install platformio
3
4 # Install ESP-IDF (handled by PlatformIO)
5 pio pkg install

```

6.2 Build Commands

```

1 # Build for ESP32-S3
2 pio run -e esp32s3
3
4 # Build for ESP32-WROOM (dev board)
5 pio run -e esp32_wroom
6
7 # Upload firmware
8 pio run -e esp32s3 --target upload
9
10 # Monitor serial output
11 pio device monitor -b 115200

```

6.3 Configuration

Edit `sdkconfig.esp32s3` for production settings:

- Enable brownout detection
- Set CPU frequency to 240 MHz
- Configure flash size (16 MB)
- Enable PSRAM if needed

7 References

- ESP-IDF MCPWM Documentation: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/peripherals/mcpwm.html>
- ESP-IDF SPI Peripheral: https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/peripherals/spi_slave.html
- FreeRTOS Task Notifications: <https://www.freertos.org/RTOS-task-notifications.html>
- DRV8243 Datasheet: TI SLVSFN4
- STAR FTP Section 16: Embedded Implementation
- STAR Software Architecture Primer: Section 12 (ESP32 Firmware)